考虑交叉口预约与充电的多机器人联合调度

EE6228 课程项目

摘要

四台机器人在两条固定的有向环路上执行取送任务，同时共用一个交叉口和数量有限的充电位。由于等待同样消耗电量，任务顺序、资源排队与电池状态相互耦合。本项目在一个有界整数调度模型中联合决定机器人分配、访问顺序、交叉口时隙、充电位选择和充电量：联合贪心（Joint Greedy）给出初始调度，CP-SAT 在此基础上搜索改进，滚动时域版本仅利用已释放的任务重新规划。返回的动作轨迹均经过先后顺序、资源容量和电池安全检查。在存档的 40 任务、10 个随机种子实验中，CP-SAT 相对 FCFS 将平均综合目标 J 降低 53.2 %，完工时间（makespan）缩短 28.3 %（FCFS 版本 75afc09，CP-SAT 版本 c87fb9f；8 个工作线程，CP-SAT 时限 120 s）。在另一个随机种子为 6 的 12 任务实例上，CP-SAT 在单线程、30 s 墙钟上限和 0.5 确定性预算下将完工时间缩短 30.4 %。短预算批量运行未发现超出联合贪心的改进。总体而言，这些结果表明，对任务、交叉口和充电进行一体化调度，有望提升固定路线多机器人系统的性能。

关键词：多机器人调度；充电调度；约束规划；交叉口预约；滚动时域；离散事件仿真。

1 引言

1.1 研究背景

多机器人系统广泛用于自动化制造和仓储环境中的运输与物料搬运。调度首先涉及任务分配，即决定由哪台机器人执行哪项任务以及机器人之间如何协同。Athira 等 [1] 综述了此类任务分配问题，以及协同与不确定性的处理方法。

在自动导引车（AGV）系统中，任务分配还会受到共享交通资源的制约。Li、Fan 和 Jia [2] 在自动化集装箱码头中将动态任务分配与无冲突路径规划相结合，要求车辆不得同时占用同一网络区域。若调度时忽略此类冲突，车辆在实际相遇时可能导致方案不可行。

电池约束带来了第二重耦合。充电设施稀缺时，充电会与运输争夺车辆时间和充电位容量；因此 Li 等 [3] 在充电位数量有限的条件下对 AGV 任务与充电进行联合调度。在这些研究的基础上，本项目研究固定路线多机器人系统中任务执行、交叉口预约与充电决策的联合调度。与模型各组成部分相关的方法将在后续相应章节（第 3 节和第 4 节）中进一步讨论。

1.2 系统概述与问题描述

本项目考虑一个多机器人取送系统：四台机器人在两条固定的有向环路上运行，每条环路分配两台机器人；每项运输任务要求一台机器人在同一环路的两个站点之间取货并送货。两条环路共用一个交叉口，交叉口同一时刻只能容纳一台机器人。机器人还共用一个最多含两个充电位的充电区，每个充电位同一时刻只服务一台机器人。

调度问题是确定由哪台机器人执行每项任务、任务的执行顺序、机器人何时通过共享交叉口，以及何时充电、充多少电。这些决策必须满足任务先后顺序、资源容量和电池安全约束。目标是最小化由完工时间、按优先级加权的迟到时间、资源等待时间和能耗构成的综合指标。

1.3 研究动机与调度难点

在本项目中，四台机器人服务两条固定环路，但共用交叉口 X 和充电区。任务分配结果直接改变机器人到达取货站、X 和充电位的时刻；由此产生的排队不仅推迟任务完成，还会持续消耗电量。因此，完工时间、迟到、等待和能耗均受同一组分配与时序决策的共同影响。

例如，当一台机器人（如 R1）接到下一项任务时，其电量看似足以完成取货和送货，却可能不足以在之后安全到达充电位，调度器就必须先安排充电。为到达充电区，机器人需要预约并通过共享交叉口；之后可能还要等待空闲的充电位，排队期间继续耗电。充电完成后，机器人必须再次预约交叉口才能返回所属环路。这些额外的移动和等待可能推迟后续任务，使其错过截止时间。因此，任务调度、资源预约和电池状态必须联合考虑。

1.4 研究问题

基于上述耦合关系，本项目研究以下问题：

- **RQ1：**在任务、地图、初始荷电状态（SOC）和离线信息均相同的条件下，与先到先服务（FCFS）和分阶段方法相比，联合决策如何影响完工时间、总迟到时间、资源等待、能耗以及归一化综合目标 J？
- **RQ2：**要求机器人在时域末端保留足以安全到达充电区的电量，会带来多大的性能代价？任务负载、充电位数量以及 A、B 两条环路之间的任务分布如何改变系统瓶颈？
- **RQ3：**当任务只有在释放后才可见时，滚动时域方法能达到怎样的调度性能，又需要多少重规划与求解开销？

1.5 贡献与报告结构

本项目建立了一个联合调度模型，涵盖任务分配、机器人访问顺序、交叉口预约、充电位选择、充电时长和 SOC 演化。离线 CP-SAT、滚动时域优化与启发式基线共用同一离散事件执行引擎和输出验证器。每次运行均输出任务输入、动作、资源预约、SOC 轨迹和求解记录，使各项结果可追溯至具体决策；存档批量实验保存了每次运行的指标和求解记录。本项目的贡献在于这种一体化建模及其在固定双环路系统上的评估。

第 2 节定义系统、输入和范围。第 3–5 节给出模型、求解方法和系统实现与验证。第 6–9 节介绍实验、分析结果、讨论局限并给出结论。附录给出完整模型、算例的任务列表与调度记录，以及关键代码节选。

2 问题定义

2.1 网络与资源配置

系统包含两条连接关系固定的单向环路（图 2.1）。环路 A 为 X → A1 → A2 → A3 → A4 → WA → X，环路 B 为 X → B1 → B2 → B3 → B4 → WB → X。机器人 R1 和 R2 只服务环路 A，R3 和 R4 只服务环路 B。WA 和 WB 是共享交叉口 X 前的等待点。充电支路为 X → H → C1/C2 → H → X，其中 H 是充电缓冲与排队位置，C1 和 C2 是可配置的充电位。

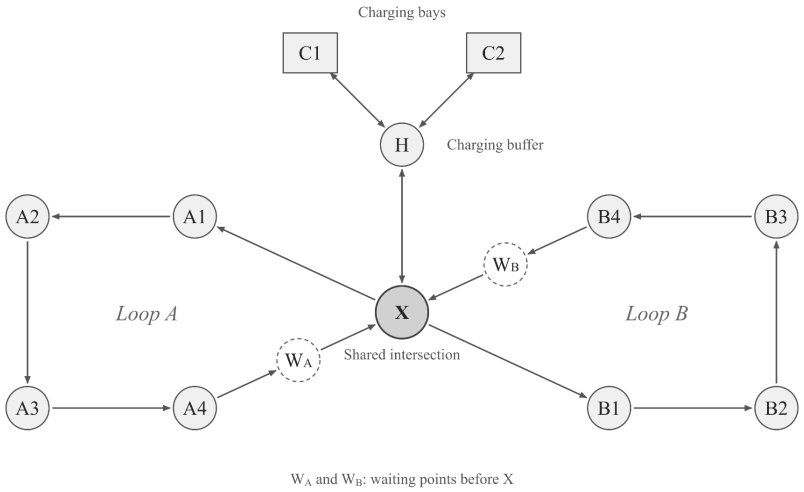


图 2.1: 网络拓扑：两条有向环路、带等待点 WA 和 WB 的共享交叉口 X，以及经 H 通往充电位 C1、C2 的充电支路。

地图共有 18 条有向边。行驶时间等于边长除以速度，移动能耗等于边长乘以单位距离能耗；地图编辑器可以修改这些参数，但不会改变连接关系。只有 X 和充电位是互斥资源，普通环路边允许多台机器人同时使用，因此模型不处理几何意义上的避碰。

2.2 任务、输入与输出

每项运输任务有八个字段：`task_id`；最早开始时间 `release_time`；同一环路上两个不同站点 `pickup` 和 `dropoff`；装货和卸货时间 `load_time`、`unload_time`；截止时间 `due_time`；以及 1–5 的整数优先级 `priority`。表 2.1 给出项目自带的样例记录，仅用于说明数据格式；第 7 节 12 任务实验的完整输入见表 B.1。

表 2.1: `examples/tasks.csv` 中的输入样例。

task_id	release_time	pickup	dropoff	load_time	unload_time	due_time	priority
T036	30	A3	A1	20	20	787	1
T012	40	B4	B2	10	20	887	2
T016	40	B4	B3	15	15	825	3

输入包括任务表、地图参数、机器人初始 SOC、安全阈值、充电位数量与充电速率、缓冲参数以及求解器选项（CP-SAT 时限、工作线程数、滚动时域）。输出包括每项任务的执行机器人和

完成时间、每台机器人的动作时间线、X 与充电位的预约区间、SOC 轨迹、验证结果以及各项指标（完工时间、加权迟到、等待、能耗、利用率和 J）。JSON 和 CSV 导出保留完整轨迹，便于审查和复现。

2.3 范围与假设

- **车队与网络。**四台机器人（每条环路两台）、一个共享交叉口、最多两个充电位；每项任务都是单一环路内的取送订单，机器人一次只携带一项任务。实验使用 20–60 项任务。
- **互斥资源。**X 和每个启用的充电位同一时刻只容纳一台机器人，相邻两次通过之间留有安全缓冲。机器人只在 WA、WB 或 H 排队等待 X 或充电位，不得停在 X 内，也不得在被占用的充电位内排队。
- **时间与能量。**各项时长是确定的。SOC 随移动、装卸、排队和线性充电变化；任务之间断电待机不耗电。任务、通过交叉口和充电访问均不可抢占，充电结束后机器人立即离开充电位。启发式规则充到 95 %，CP-SAT 则自行选择充电量。
- **信息条件。**启发式规则和离线 CP-SAT 可以看到完整任务表，但不会在释放前开始任务；滚动 CP-SAT 只看到已释放的任务，并保持已提交的动作不变。手动命令回放只执行和检查外部计划，不参与算法比较。
- **不在范围内。**非线性充电、多个交叉口、可变车队规模、连续运动学和避障。机器人无需返回起点；末端储备只保证最后一项任务之后仍能到达充电位。

3 数学模型

本节在第 2.3 节界定的范围内建立调度决策的数学模型。每项任务依次包括空驶至取货点、装货、载货行驶和卸货。

3.1 集合、参数与决策变量

表 3.1 和表 3.2 定义了符号。时间单位为秒；SOC 和能量以电量百分点计。下标 0 表示每台机器人的虚拟初始顶点。

表 3.1: 集合与参数。

符号	含义
I, K, K_i	任务集合、机器人集合，以及可执行任务 i 的机器人集合
P, Q_k	启用的充电位，以及机器人 k 的候选充电访问
r_i, d_i, p_i	任务 i 的释放时间、截止时间和优先级
L_i, U_i, g_h	装货和卸货时间，以及每次装卸的 SOC 消耗
τ_{uv}, g_{uv}	有向边 (u, v) 的行驶时间和 SOC 消耗
ρ_w, ρ_c	排队耗电速率和充电速率（每秒 SOC 百分点）
$S_{\min}$, β, δ	安全 SOC 下限、缓冲参数和通过间隔 $\delta = 2\beta$
$\hat{R}_C$, $\hat{R}_T$, $\hat{R}_W$, $\hat{R}_E$	完工时间、加权迟到、等待和能耗的归一化参考值

表 3.2: 决策变量。

类别	符号	含义
分配与顺序	x_{ik}, a_{kab}	任务分配；机器人 k 在访问 a 之后访问 b 的弧变量
任务与访问时间	s_i, c_i, s_{kn}, f_{kn}	任务开始与送达完成时间；机器人 k 第 n 个访问的开始和结束
迟到与完工时间	T_i, $C_{\max}$	任务迟到时间和最后一次送达的完成时间
交叉口预约	e_{kr}, l_{kr}, w_{kr}	通过的进入时间、离开时间和之前的排队时间
充电决策	y_{kq}, b_{kqp}, u_{kq}, D_{kq}	充电访问是否启用、充电位选择、开始时间和时长
电池状态	S^H, S^-, S^+	到达 H 时、开始充电前和充电后的 SOC
等待与能耗	w^C, W_X, W_C, E	充电排队；交叉口与充电的总排队；正的 SOC 消耗总量

3.2 分配与时序约束

模型引入虚拟初始顶点，通过 `AddCircuit` 约束对每台机器人获配的任务和可选充电访问进行排序。任务顶点当且仅当任务被分配时激活；充电顶点在该访问启用时激活。未激活的顶点使用自环，虚拟初始顶点的自环表示机器人闲置。分配、弧、激活和充电位变量均为二元变量。被选中的弧满足

$$\sum_{k\in K_i} x_{ik} = 1, \qquad s_i \ge r_i, \qquad a_{kab} = 1 \;\Rightarrow\; s_{kb} \ge f_{ka} \quad (b \neq 0). \tag{3.1}$$

其中 s_i 包含空驶至取货点的时间，c_i 为卸货完成时间，f_{ka} 为前一个访问的结束时间。若任务 i 分配给机器人 k，其完成时间满足式 (3.2)：空驶时间由所选前驱顶点决定，载货行驶沿固定的取送路线进行，求和项累加集合 $\mathcal{R}_{ki}$（所选空驶和载货路径上的交叉口通行）中允许的资源排队时间。闭合到虚拟顶点的弧不含返程行驶，因此完工时间止于最后一次送达。

$$c_i = s_i + \tau^{\text{empty}}_{ki} + L_i + \tau^{\text{loaded}}_{i} + U_i + \sum_{r\in\mathcal{R}_{ki}} w_{kr}. \tag{3.2}$$

3.3 资源容量与电池安全

每次选中的交叉口通行都在同一个 `NoOverlap` 约束中预约区间 $[e_{kr},\, l_{kr}+\delta)$，其中 e_{kr} 和 l_{kr} 为进入和离开时间，$\delta = 2\beta$；默认缓冲参数为 2 s，因此相邻两次通行之间间隔 4 s。这是对交叉口的按时间预约，思路与自动驾驶车辆基于预约的交叉口管理 [4] 相同。机器人在 WA、WB 或 H 等待，不得停在 X 内。

每次选中的充电访问先在 H 排队，进入一个选定的充电位，不间断地充电，然后立即返回 H；充电位区间同样互不重叠。充电位选择满足 $\sum_p b_{kqp} = y_{kq}$。设到达 H 的时间为 h，则充电开始于 $u = h + w^C + \tau_{Hp}$，访问结束于 $u + D + \tau_{pH}$。充电量和充电时长都是决策变量，这与充电位数量有限时采用灵活充电策略的 AGV 调度 [3] 以及将充电时长作为决策变量的模型 [5] 一致；启发式规则则固定充到 95 %。

SOC 随移动、装卸和资源排队而下降，任务之间的断电待机不耗电。对于不含充电、包含 n_h 次装卸的路段，

$$S_{\text{out}} = S_{\text{in}} - \sum_{(u,v)} g_{uv} - n_h\, g_h - \rho_w \sum w, \tag{3.3}$$

系统在充电前以及每个动作之后校验该式，并要求电量保持在 $[S_{\min}, 100]$ 之内。对于到达 H 时 SOC 为 S^H、排队时间为 w^C、充电时长为 D 的充电访问，安全到达、补电和离开要求

$$S^- = S^H - \rho_w w^C - g_{Hp} \geq S_{\min}, \qquad S^+ = S^- + \rho_c D \leq 100, \qquad S^+ - g_{pH} \geq S_{\min}. \tag{3.4}$$

默认的末端储备要求每台执行过任务的机器人在最后一次送达后保留足够电量，以便到达充电位并承受保守估计的排队，其中包含为其他三台机器人各一次充电预留的余量；整数模型额外加 0.5 个百分点的余量，解码后的计划还要对照实际预约日历检查。消融版本 `cpsat-nores` 仅移除这一末端储备要求，仍保留执行过程中的电池安全约束；由于可行域更大，该版本予以单独评估。滚动时域优化则始终保留末端储备。

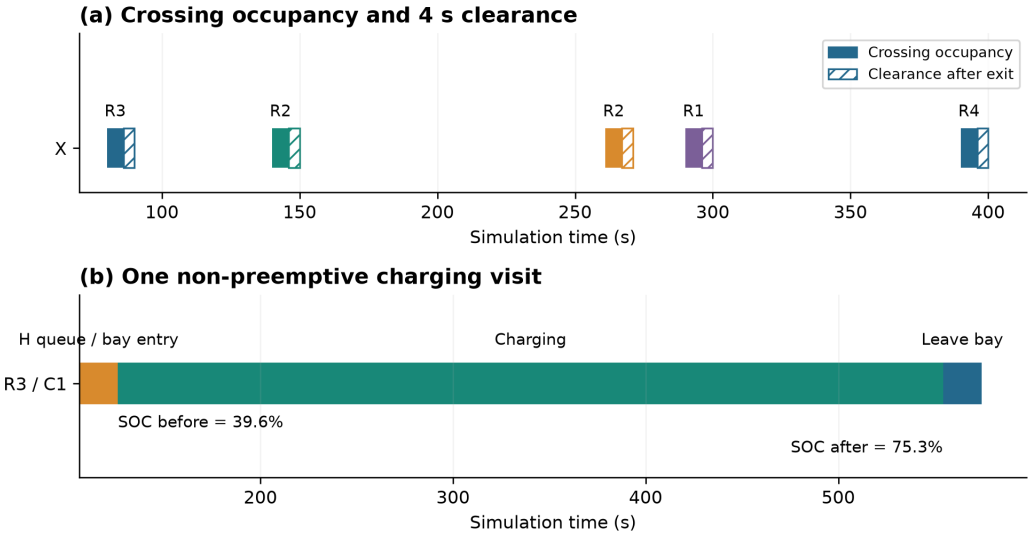


图 3.1: 交叉口预约与单次充电访问（本地 12 任务运行，随机种子 6，安全 SOC 25 %）。实心的通行区间不含带阴影的 4 s 间隔；充电访问包括从 H 驶入、充电及充电后立即驶离。

3.4 目标函数与模型范围

优化器最小化由完工时间、迟到、等待和能耗组成的综合目标。记 $T_i = \max(0, c_i - d_i)$、$C_{\max} = \max_i c_i$，交叉口和充电的总排队时间为 W_X 和 W_C，正能耗总量为 E，则

$$\min J = \frac{1}{4}\left(\frac{C_{\max}}{\hat{R}_C} + \frac{\sum_i p_i T_i}{\hat{R}_T} + \frac{W_X + W_C}{\hat{R}_W} + \frac{E}{\hat{R}_E}\right). \tag{3.5}$$

能耗与延误一同进入目标函数，这与多载 AGV 的多目标调度中兼顾装卸延误与能耗的做法 [6] 类似。参考值通常取同一实例上 FCFS 的结果，分母下限分别为 1 s、60 s、60 s 和 1 个 SOC 百分点。若 FCFS 失败，则依次尝试分阶段和截止时间优先，最后才只用下限作为参考值；因此 FCFS 的得分不一定恰好为 1。能耗不计补电，加权迟到与 CSV 中未加权的 `total_tardiness` 字段不同。电池安全是硬约束，不能用目标函数中的其他项补偿。

CP-SAT 使用 1 ms 的时间网格、保守的整数近似和有限数量的可选充电时段。状态 OPTIMAL 针对这一有界整数模型，所有候选解在返回前都经过解码和验证。

4 求解方法

4.1 基线策略与联合贪心调度

所有自动方法共用同一执行器、地图、能耗规则和指标。表 4.1 总结了它们的决策规则。FCFS 在 SOC 低于 20 % 时先充电再执行任务；接受的任务还必须通过完成检查和末端储备检查。

表 4.1: 基线调度策略。

策略	决策规则	比较目的
FCFS	按释放时间处理；选择预计最早完成的机器人，计入排队、充电和储备检查	简单可执行的参照
分阶段（Staged）	先按名义行驶时间固定分配和顺序，再加入预约和必要的充电	衡量分阶段规划的代价
截止时间优先（Deadline）	按截止时间排序,优先级打破平局,并检查资源与电量可行性	比较截止时间优先规则
联合贪心（Joint Greedy）	对最多八项待处理任务和可执行的机器人打分，提交最优的可行候选	联合启发式，兼作 CP-SAT 初始调度

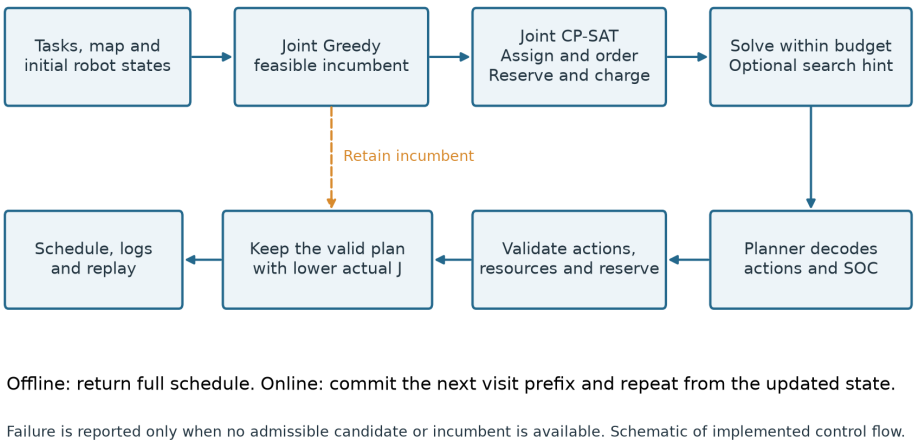


图 4.1: 联合规划与验证流程：求解器、解码器、验证器以及向初始解的回退。所有自动策略共用同一套执行规则。

联合贪心按释放时间顺序考虑最多八项待处理任务及两台可执行的机器人。每个候选方案都在状态副本上评估：尝试执行该任务，检查任务后的储备，必要时预先插入充电。不可行的候选会被丢弃，不影响已提交的调度。算法选取使局部得分

$$\text{score}_{ik} = c_i + 3\,p_i T_i + 0.5\,\Delta W_i + 2\,\Delta E_i \tag{4.1}$$

最小的候选方案，得分相同时按任务和机器人编号打破平局；增量等待 ΔW_i 和正能耗 ΔE_i 包含插入的充电访问。该得分使用原始单位和固定系数，而不是归一化的 J。它能快速给出联合初始调

度。各启发式规则均可访问完整任务列表，但实际执行不早于释放时间。若没有候选通过检查，策略会报告未找到安全计划。此外，也可采用基于学习的调度器（例如 Chang 等 [7] 的分层强化学习调度方法）；本项目保留显式规则，以便每个决策都能对照约束进行检查。

4.2 离线联合优化

机器–AGV 集成调度既有混合整数规划模型 [8]，也有显式处理运输任务是否存在的约束规划模型 [9]。本项目同样仅在访问被选中时将其激活，并采用 Google OR-Tools 的 CP-SAT 求解器 [10] 求解模型。CP-SAT 融合了惰性子句生成、线性松弛以及包括大邻域搜索在内的并行组合搜索 [11]，十分契合本问题：任务分配与访问选择均为布尔变量，可选区间、`AddCircuit` 和 `NoOverlap` 可以直接表达访问顺序和资源容量，而线性的时间与 SOC 约束则将这些选择与可行执行联系起来。联合贪心提供初始物理调度；当该调度可投影至时间网格上时，还作为 `AddHint` 搜索提示，引导搜索但不固定决策。有效的初始调度同时独立保存在求解器之外。

求解所得回路通过仿真器的规划器解码，使用模型给出的访问顺序、通行时间、充电位和充电目标。实际时长和 SOC 变化根据地图重新计算，而不是直接取自整数变量；解码器还会检查生成的动作和启用的末端储备。最终返回的计划取有效解码候选与合格启发式初始解中实际 J 较低者，因此 CP-SAT 返回的计划不会劣于联合贪心。

每次运行都会保存求解状态、界、间隙（gap）、解码后的目标值和回退原因，并与返回调度的验证结果分开记录。总运行时间包括初始规划、建模、求解、解码和验证，而求解器墙钟时间只覆盖搜索。

4.3 滚动时域优化

在决策时刻 t，滚动时域优化仅可获取已释放且未提交的任务。与仓储 AGV 动态任务的滚动分配 [12] 类似，计划随信息到达而重新计算：默认每 60 s 一次，窗口约为每台机器人 600 s 的工作量。任务按截止时间、优先级和编号排序。窗口限制的是考虑的工作量，而不是完成期限：前往取货点的行驶、排队或充电都可能使计划超出窗口。机器人当前状态和已提交的预约构成下一次求解的初始条件。已有研究同样采用滚动时域再优化在执行过程中调整 AGV 通过共享交叉口的先后顺序 [13]，这与本文滚动时域方法修正通行时隙和充电时隙的方式相近。

每台机器人仅提交在下一个决策时刻之前启动的访问前缀。已提交的访问整体冻结，包括延续到该时刻之后的动作和预约；执行中的任务和充电不会被中断，未提交的任务之后仍可重新分配。每次求解都用当前已知的任务和状态进行归一化；全实例参考值只用于事后评分，因此未释放的订单既不影响当前决策，也不影响目标系数。

5 系统实现与验证

5.1 共享执行与可追溯性

LoopLab 由 Python 后端和浏览器回放前端组成。`simulator.py` 维护机器人状态、预约日历、动作生成、指标和验证；`optimizer.py` 提供离线联合优化和滚动时域优化；`experiments.py` 复用实例并汇总结果。前端只可视化返回的轨迹，不另行运行调度器。动作记录机器人和任务编号、

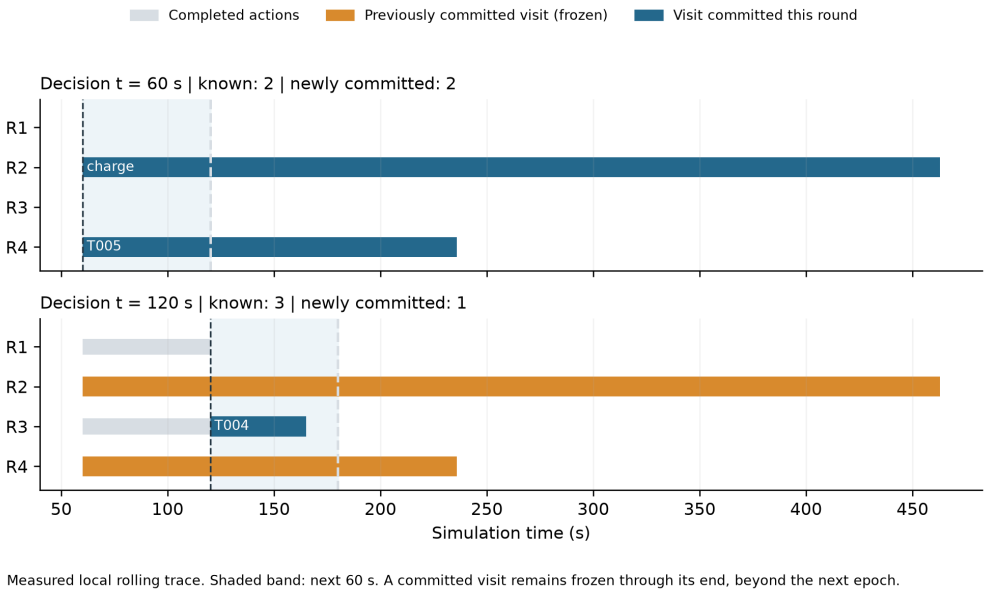


图 4.2: 一次本地运行中的滚动时域提交，决策时刻为 60 s 和 120 s。第一轮提交的访问在下一个决策时刻之后仍保持冻结。

起止节点、开始和结束时间、SOC、能耗和资源；事件、甘特图、电量曲线以及 CSV/JSON 导出都对应同一次运行，因此汇总结果可以追溯到具体动作和预约。

严格命令回放为带绝对开始时间的 `execute_task`、`move_to`、`charge` 和 `wait` 命令提供单独接口。它会拒绝时间、资源和电量冲突，而不是自动排队或插入充电；正在充电的机器人在收到明确的移动命令前一直停在充电位上，被省略的任务保持未执行。因此手动计划不参与自动算法的排名。

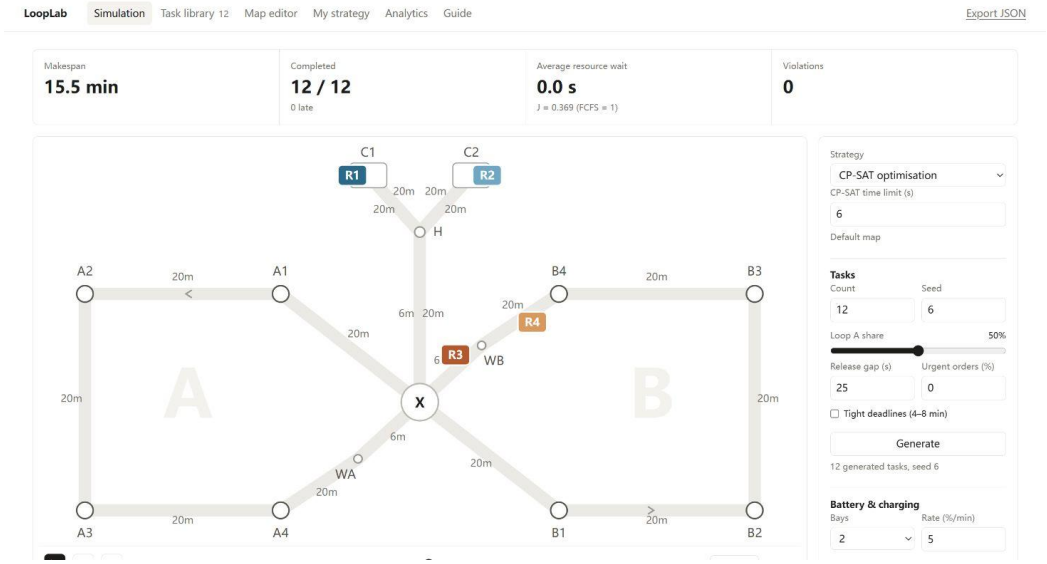


图 5.1: 本地仿真器回放：另一次 12 任务运行的浏览器地图、机器人位置和汇总指标。其交互式求解设置与第 7 节的受控算例不同，定量比较使用保存的算例输出。

5.2 独立验证

验证器根据输出的任务、动作、地图和预约重新检查正确性，包括：每项任务只执行一次、释放时间和先取后送的先后关系、机器人动作的连续性与不重叠、合法的有向行驶、载量、X 和充电位的占用与间隔、合法的等待位置，以及 SOC 的连续性和上下界。根据动作重建占用情况还能发现缺失或不一致的预约记录。未通过检查的候选会被拒绝，即使优化器报告其可行。

单任务校验算例设置如下：从 A1 到 A2、释放时间 0、截止时间 600 s、优先级 1、装卸各 15 s、初始 SOC 80 %。默认 20 m 的边耗时 20 s、消耗 2 个 SOC 百分点，每次装卸消耗 0.5，因此完成时间为 50 s，最终 SOC 为 77 %。使用 FCFS 参考值及其下限，$J = \frac{1}{4}\left(\frac{50}{50} + \frac{0}{60} + \frac{0}{60} + \frac{3}{3}\right) = 0.5$。表 5.1 将该手算结果与一次新的程序运行进行对比。若将最终 SOC 伪造为 78 %，虽高于安全下限，仍无法通过一致性检查。

表 5.1: 手算结果与程序输出。

指标	手算	程序输出
完成时间（s）	50.0	50.0
能耗（SOC 百分点）	3.0	3.0
最终 SOC（%）	77.0	77.0
目标值 J	0.500	0.500
约束违反数	0	0

目标值计算与可行性检查分开进行：验证通过后，结果构建函数重新计算最后一次送达时间、任务迟到与优先级、资源排队和正的动作能耗，再套用保存的归一化参考值。仓库中的测试检查这一计算，验证器测试则拒绝时间、SOC 或占用被篡改的轨迹。验证器状态独立于规划器，但与之共用地图和规则计算函数，因此通过验证表明调度符合所编码的模型。实现代码和实验数据见 `https://github.com/Eth4nMurpHy/robot-simulator`（代码快照 `9bf0b70`）。

6 实验设置

6.1 评估问题与数据生成

RQ1 评估联合调度相对分阶段方法和单一规则基线能否改善完工时间、迟到、等待和能耗；RQ2 考察末端储备和资源敏感性；RQ3 考察在线求解质量和重规划开销。实验使用针对四机器人双环路系统生成的合成任务。

对于 n 项基础任务，释放时间在 $[0, 12n)$ 秒内以 10 s 为步长均匀抽样。取货站和送货站互不相同且位于同一环路。装货和卸货各取 10、15 或 20 s，截止时间为释放时间加 480–900 s。优先级 1、2、3 的概率分别为 50 %、25 % 和 25 %。紧急订单和每台机器人 50–100 % 的初始 SOC（保留一位小数）由独立的随机流生成，因此加入紧急订单不会改变基础工作负载。

合成数据代表反复出现的运输调度决策：同环路订单符合固定导轨布局；释放时间形成相互竞争的请求；装卸时长和截止余量改变工作量和紧迫程度；不同的初始 SOC 产生充电需求。

6.2 实验因素与计算环境

基础设置为 40 项任务、两个充电位、10 % 安全阈值、两区域分配概率相等且无紧急订单。每次只改变一个因素（表 6.1）。每种配置使用随机种子 1–10，各策略输入相同。10 种不同配置 × 10 个种子 × 7 种策略得到 700 条去重后的运行记录；CSV 中的 1 050 行包含在不同实验组中重复出现的基础设置结果。

表 6.1: 实验因素与求解预算。

因素	取值或预算
任务数量	20、40、60；释放窗口 $12n$ 秒
充电位数量	1 或 2
区域不均衡	期望 A:B 比例为 1:1、2:1、3:1
安全 SOC	10 %、20 %、30 %
紧急订单	额外增加 0 %、10 %、20 %；截止余量 180–360 s；优先级 4 或 5
求解预算	离线：120 s，8 个工作线程。滚动：单线程，每次求解确定性预算 0.1，墙钟上限 30 s；每 60 s 重规划，前瞻约 600 s

存档实验使用 Python 3.13 和 OR-Tools 9.15.6755。启发式规则和滚动时域优化基于提交版本 75afc09 在双 vCPU 的 Colab 机器上运行；离线 CP-SAT 使用配备 Intel Xeon 2.20 GHz 处理器的高内存 Colab 机器。无末端储备的运行使用 75afc09，保留末端储备的运行使用 c87fb9f，SOC 余量为 0.5 个百分点。各方法所用硬件不同，运行时间仅供参考。滚动时域方法的预算 0.1 以确定性工作量计，而非秒。

本地演示运行使用快照 9bf0b70、Windows 11、Python 3.12.14 和 OR-Tools 9.15.6755，处理器为 AMD Ryzen 7 7840H；其随机种子、预算和样本量与第 7 节的结果一并给出。历史观测与本地观测分开报告，表 5.1 的单任务检查也使用同一快照。

6.3 指标、统计与公平比较

指标包括 J、完工时间、未加权和按优先级加权的迟到、迟到任务比例、资源等待、能耗、充电次数、运行时间和违反数。平均资源等待为 $(W_X + W_C)/|I|$，不含断电待机；能耗为正的 SOC 消耗之和，不计补电。表格按需报告均值和样本标准差；本地图表显示中位数和各随机种子的结果。成对目标降幅为 $1 - J_s/J_{\mathrm{FCFS}}$，正值表示改进；它使用 FCFS 的实际 J，由于分母下限，该值可能小于 1。

启发式规则和离线 CP-SAT 预知未来任务但遵守释放时间，而滚动时域方法仅获知已释放的任务。主要比较均保留末端储备；无末端储备的 CP-SAT 仅作为消融实验。全部 700 条存档记录均报告成功且违反数为零。

7 结果与分析

存档实验提供了第 6 节所述的 10 个种子、7 种策略的比较。本地图表使用 3 个种子、4 种配置和 5 种离线策略，共 60 次运行，另有一个包含 FCFS、CP-SAT 和滚动时域计划的独立 12 任务算例。存档实验提供总体比较，本地轨迹用于解释具体决策。

7.1　总体调度性能

表 7.1 给出存档基础设置的结果：40 项任务、两个充电位、10 % 安全 SOC。保留末端储备的 CP-SAT 平均 $J = 0.468$，FCFS 为 1.000，降低 53.2 %。平均完工时间从 3 233 s 降至 2 317 s（28.3 %），能耗从 443.2 降至 358.8 个 SOC 百分点（19.0 %）。

表 7.1: 存档基础设置的 10 个种子结果。迟到、充电排队和能耗为均值；迟到未加权，能耗不计补电。

方法	J（均值 ± 标准差）	完工时间（s）	迟到（s）	充电排队（s）	能耗（百分点）
FCFS	1.000 ± 0.000	3233 ± 384	26 028	1 437	443.2
分阶段	1.067 ± 0.156	3441 ± 543	27 264	1 487	449.0
截止时间优先	1.003 ± 0.240	3210 ± 499	25 994	1 290	438.3
联合贪心	0.793 ± 0.154	3021 ± 479	17 826	909	386.4
CP-SAT	0.468 ± 0.062	2317 ± 477	9 934	0	358.8
CP-SAT（无储备）	0.349 ± 0.072	1732 ± 542	5 032	0	316.1
滚动 CP-SAT	0.636 ± 0.136	2741 ± 531	15 651	255	388.9

联合贪心使平均 J 降低 20.7 %，而分阶段和截止时间优先相对 FCFS 平均几乎没有优势：先固定任务分配再预约资源，可能把充电冲突留到执行阶段被动消解。滚动时域方法和无末端储备的 CP-SAT 因使用的信息或约束不同而单独列出。

由于每个实例都用各自的 FCFS 值归一化，并对较小的分项设置下限，FCFS 的得分可能略低于 1，部分 20 任务的本地运行就是如此。

7.2　求解预算与本地验证

存档的离线 CP-SAT 使用 120 s 和 8 个工作线程，返回的解均为 FEASIBLE。在基础设置下，其总运行时间平均为 120.67 s，联合贪心为 0.29 s：CP-SAT 以更多的计算时间换取更好的调度。

本地检查每次求解使用单个 CP-SAT 工作线程、0.15 的确定性预算和 6 s 墙钟上限。图 7.1 给出 40 任务基础设置下每个种子的结果。12 次本地 CP-SAT 运行的 J 均与联合贪心相同，其中 9 次明确通过回退保留了初始解，说明该短预算不足以改进初始调度。全部 60 个批量输出、三个算例计划以及浏览器导出的计划都通过了验证，仓库测试套件的 69 个测试也全部通过。

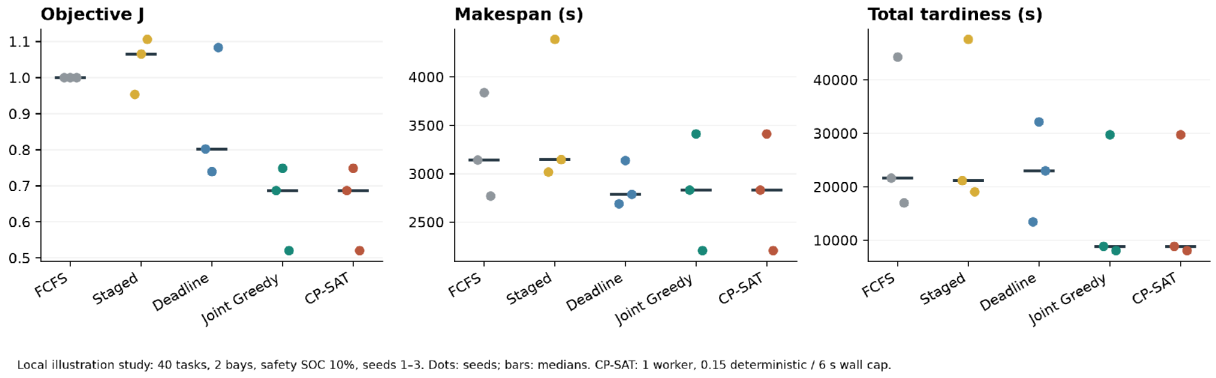


图 7.1: 本地算法比较：3 个种子、40 项任务、两个充电位、安全 SOC 10 %。圆点为单次运行，短横线为中位数。在短预算下 CP-SAT 与联合贪心重合。

求解记录解释了图 7.1 中的重合。9 次回退运行的状态为 UNKNOWN，因为在预算内没有找到 CP-SAT 的可行解，但它们仍返回经过验证的联合贪心计划。其余 3 次运行均为 20 项任务，状态为 FEASIBLE，报告的间隙约为 36–64 %，返回的 J 仍与初始计划相同。本地 12 任务的离线解同样为 FEASIBLE，间隙约 56 %。

7.3　调度算例与瓶颈分析

本地算例包含 12 项任务，随机种子 6，释放间隔 25 s，两个充电位，安全下限 25 %。R1–R4 的初始 SOC 分别为 50.1、53.9、52.2 和 80.8 %。离线 CP-SAT 使用单线程、0.5 的确定性预算和 30 s 墙钟上限，实测总运行时间为 2.62 s。图 7.2 中 FCFS 与 CP-SAT 使用相同的时间轴，表 7.2 还包括同一输入下的滚动时域方法；完整输入见表 B.1。

表 7.2: 本地 12 任务算例。

方法	J	完工时间（s）	迟到（s）	充电排队（s）	最低 SOC（%）
FCFS	1.000	1673.2	2518.4	634.4	35.4
CP-SAT	0.447	1165.2	275.2	0.0	25.9
滚动 CP-SAT	0.505	1237.6	1011.0	0.0	33.4

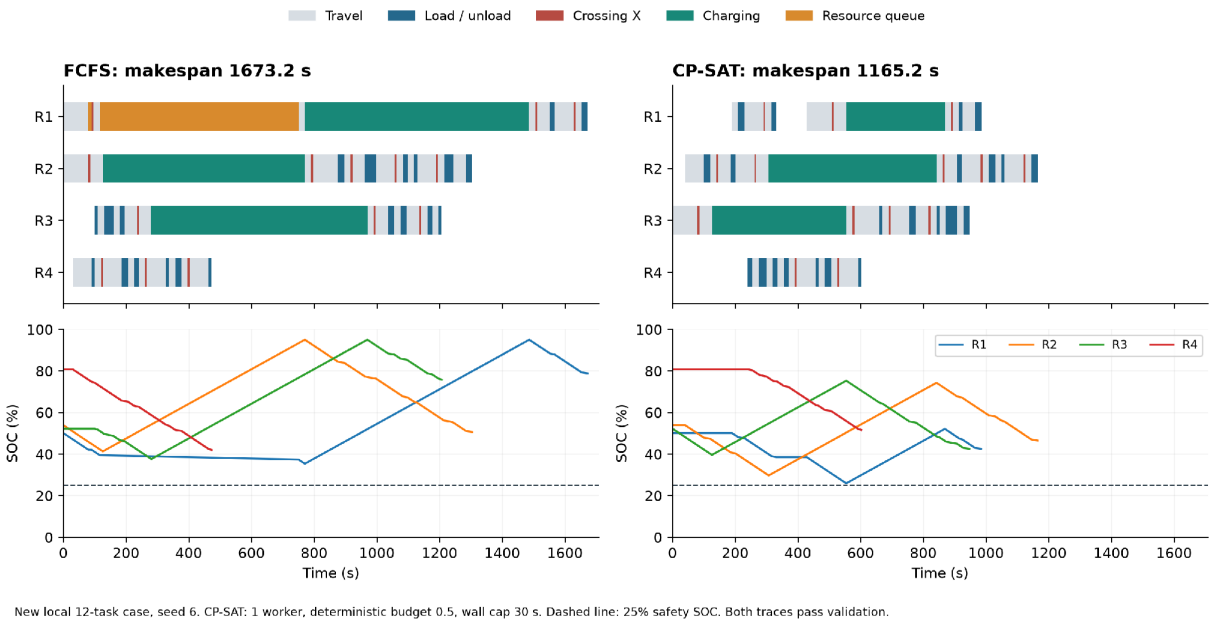


图 7.2: 本地调度与 SOC 比较。上方为机器人动作，下方为 SOC；虚线为 25 % 的安全下限。两份调度都保留了末端储备并通过验证。

在 FCFS 下，R1 从 116.0 s 到 750.4 s 在 H 等待，充电排队 634.4 s，之后的充电一直持续到 1 486.2 s。CP-SAT 先让 R3 从 126.0 s 充到 554.4 s，随后在 554.4 s 立即让 R1 进入充电位；R1 在 868.8 s 充电结束。CP-SAT 的三个充电目标分别为 75.3、74.3 和 52.1 %，而不是 95 %。上述调整消除了充电排队并紧凑了整体调度：完工时间缩短 30.4 %，未加权迟到减少 89.1 %。能耗几乎不变（160.548 对 160.400 个 SOC 百分点），因此改进来自更好的时序与充电协调，而非节能。平均资源等待从每项任务 53.7 s 降为零，其中既包括 FCFS 的充电排队，也包括其 10 s 的交叉口排队。

7.4　末端储备与资源敏感性

在存档基础设置下，去掉末端储备使平均 J 从 0.468 降至 0.349，完工时间从 2 317 s 降至 1 732 s。执行过程中的电池安全仍然强制满足，只去掉了批次结束后的末端储备；收益来自更大的可行域，代价是批次结束后机器人可能无法到达充电位。

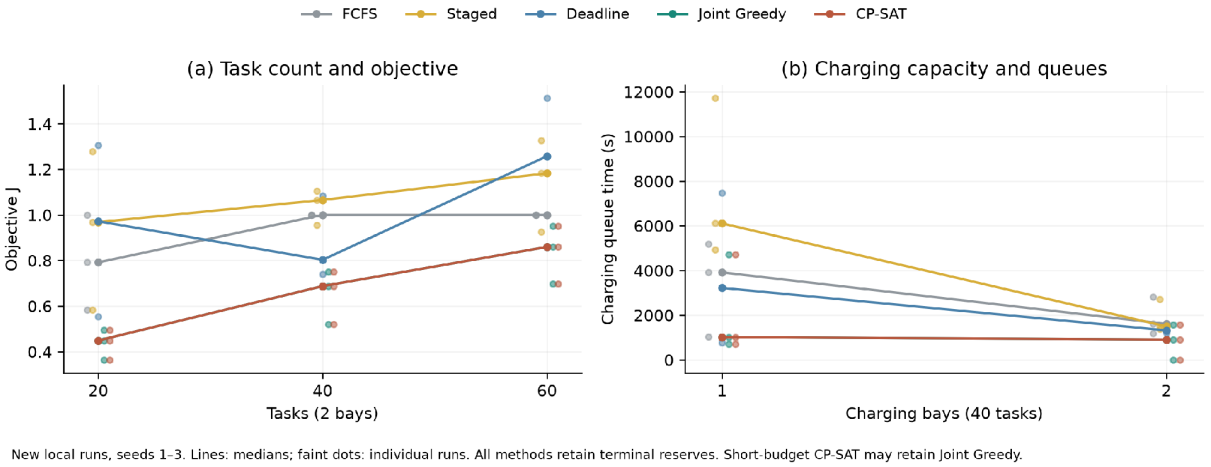


图 7.3: 本地工作量与充电敏感性。折线为种子 1–3 的中位数，浅色圆点为单次运行。左：两个充电位下的任务数量；右：40 项任务下的充电位数量。

图 7.3 显示，本地联合贪心的 J 中位数从 20 项任务时的 0.449 升至 40 项时的 0.687 和 60 项时的 0.860。在 40 项任务下，将充电位从两个减为一个，FCFS 的充电排队中位数从 1 604.6 s 增至 3 913.8 s，而联合贪心只从 910.4 s 增至 1 012.2 s；可见调度方式会改变资源短缺的影响程度。

存档的敏感性结果覆盖了本地检查之外的条件。期望的 A:B 任务比例从 1:1 变为 3:1 时，保留末端储备的 CP-SAT 平均完工时间从 2 317 s 增至 3 010 s。安全下限从 10 % 提高到 30 %，其迟到任务比例从 32.5 % 增至 48.0 %；增加 20 % 的紧急任务使平均完工时间增至 2 820 s，迟到比例增至 48.8 %。因此，即使每个返回的计划都可行，约束和工作量仍会限制配送表现。各因素分别考察不同维度：充电位数量考察资源可用性，区域不均衡考察工作负载的集中程度，安全 SOC 则收窄了可用电量范围。

7.5　在线重规划

在本地 12 任务输入上，滚动时域 CP-SAT 进行了 18 次重规划，没有发生回退。总运行时间为 5.40 s，其中求解器墙钟时间为 5.10 s；完工时间为 1 237.6 s，J 为 0.505。其完工时间虽晚于离线 CP-SAT，但只能在任务释放后获取任务。存档基础设置下的平均 J 为 0.636，同样介于 FCFS 与离线 CP-SAT 之间。因此，滚动 CP-SAT 只使用已释放的任务，就保留了离线方法的大部分收益，每次重规划约 0.3 s。

8 讨论

8.1 结果解读

存档实验和本地实验都表明，在所测试的系统中，对任务分配、交叉口预约和充电决策进行联合调度确有裨益。联合贪心相对 FCFS 将平均综合目标降低 20.7 %，并通过协调的资源与电量检查给出可行调度。离线 CP-SAT 在存档的搜索预算下取得了进一步改进，其中灵活的充电量也起到了作用。

算例轨迹揭示了一个具体瓶颈：固定充到 95 % 可能长时间占用充电位，进而推迟其他机器人的作业。部分充电和任务重排可以消除这种延误。

短预算结果也说明了计算限制的重要性：CP-SAT 可能直接返回经过验证的联合贪心调度而没有改进。因此，联合贪心是实用的初始解和回退方案，而在搜索时间充足时，CP-SAT 能够提升调度质量。

8.2 局限性

本研究针对四机器人固定路线系统和合成工作负载。验证覆盖了所建模的资源与电池约束，但不涉及运动层面的避碰，后者需要如文献 [2] 那样的冲突模型。模型假设电池充放电行为确定且呈线性；能耗的不确定性可借助随机模型处理，例如 Ricard 等 [14] 针对电动公交提出的机会约束电池管理。更大的车队、更多的随机种子以及对部分充电的单独测试是后续可做的工作。

9 结论

本项目针对固定双环路系统实现了任务、交叉口与充电的联合调度。在存档基础实验中，保留末端储备的 CP-SAT 相对 FCFS 将平均 J 降低 53.2 %，完工时间缩短 28.3 %。本地算例表明，更短的调度来自充电排队的消除；短预算检查则说明 CP-SAT 需要足够的搜索时间才能改进联合贪心。

末端储备以部分批次性能换取每台机器人在最后一项任务后仍能到达充电位。滚动时域规划在冻结已提交动作的前提下处理已释放的任务，并保留了离线方法的大部分收益。后续工作可将模型扩展到更大的车队、不确定的能耗和实际部署。

参考文献

[1] Athira K. A., Divya Udayan J., and U. Subramaniam, “A systematic literature review on multi-robot task allocation,” *ACM Computing Surveys*, vol. 57, no. 3, Art. no. 68, 2025, doi:10.1145/3700591.

[2] S. Li, L. Fan, and S. Jia, “A hierarchical solution framework for dynamic and conflict-free AGV scheduling in an automated container terminal,” *Transportation Research Part C: Emerging Technologies*, vol. 165, Art. no. 104724, 2024, doi:10.1016/j.trc.2024.104724.

[3] Y. Li, B. Ren, X. Wen, and S.-H. Chung, “An adaptive large neighborhood search method for the AGV scheduling problem with a limited number of chargers,” *International Journal of Production Research*, vol. 63, no. 15, pp. 5749–5781, 2025, doi:10.1080/00207543.2025.2462670.

[4] K. Dresner and P. Stone, “A multiagent approach to autonomous intersection management,” *Journal of Artificial Intelligence Research*, vol. 31, pp. 591–656, 2008, doi:10.1613/jair.2502.

[5] Y. Li, X. Wen, S. Zhou, and S.-H. Chung, “Smart automated guided vehicle scheduling with flexible battery management: A new formulation and an exact approach,” *Computers & Operations Research*, vol. 183, Art. no. 107156, 2025, doi:10.1016/j.cor.2025.107156.

[6] X. Huo, X. He, Z. Xiong, and X. Wu, “Multi-objective optimization for scheduling multi-load automated guided vehicles with consideration of energy consumption,” *Transportation Research Part C: Emerging Technologies*, vol. 161, Art. no. 104548, 2024, doi:10.1016/j.trc.2024.104548.

[7] X. Chang, X. Jia, and H. Hu, “Energy-efficient and self-adaptive AGV scheduling approach based on hierarchical reinforcement learning for flexible shop floor,” *Computers & Industrial Engineering*, vol. 205, Art. no. 111140, 2025, doi:10.1016/j.cie.2025.111140.

[8] Y. Yao, Q. Liu, L. Fu, X. Li, Y. Yu, L. Gao, and W. Zhou, “A novel mathematical model for the flexible job-shop scheduling problem with limited automated guided vehicles,” *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 7449–7462, 2025, doi:10.1109/TASE.2024.3356255.

[9] Y. Yao, Q. Liu, X. Li, and L. Gao, “Constraint programming for AGV and machine integrated scheduling problem in flexible manufacturing system,” *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 2378–2390, 2026, doi:10.1109/TASE.2025.3650678.

[10] L. Perron and F. Didier, *CP-SAT*, Google OR-Tools, version 9.15.6755, 2026. [Online]. Available: `https://developers.google.com/optimization/cp/cp_solver/`

[11] L. Perron, F. Didier, and S. Gay, “The CP-SAT-LP solver,” in *Proc. 29th Int. Conf. Principles and Practice of Constraint Programming (CP 2023)*, LIPIcs, vol. 280, pp. 3:1–3:2, 2023, doi:10.4230/LIPIcs.CP.2023.3.

[12] J. Xin, Q. Yuan, A. D’Ariano, G. Guo, Y. Liu, and Y. Zhou, “Dynamic unbalanced task allocation of warehouse AGVs using integrated adaptive large neighborhood search and Kuhn–Munkres algorithm,” *Computers & Industrial Engineering*, vol. 195, Art. no. 110410, 2024, doi:10.1016/j.cie.2024.110410.

[13] A. Berndt, N. van Duijkeren, L. Palmieri, A. Kleiner, and T. Keviczky, “Receding horizon re-ordering of multi-agent execution schedules,” *IEEE Transactions on Robotics*, vol. 40, pp. 1356–1372, 2024, doi:10.1109/TRO.2023.3344051.

[14] L. Ricard, G. Desaulniers, A. Lodi, and L.-M. Rousseau, “Chance-constrained battery management for electric bus scheduling,” *European Journal of Operational Research*, vol. 336, no. 3, pp. 940–952, 2027, doi:10.1016/j.ejor.2026.05.046.

A 完整模型

实现中使用 $V_k = \{0\} \cup I_k \cup Q_k$，其中 I_k 为机器人 k 所在环路上的任务集合，Q_k 为其可选充电时段。任务的存在变量为 $z_{ki} = x_{ik}$，充电访问为 $z_{kq} = y_{kq}$，且 $z_{k0} = 1$；分配 x、弧 a、激活 y 和充电位 b 均为布尔变量。下列各式以秒和 SOC 百分点为单位；在内部，时间变量是 $[0, \bar{H}]$ 内的整数刻度，节点 SOC 和充电目标位于 $[US_{\min}, 100U]$ 内。虚拟初始状态固定了初始可用时间、位置和 SOC。

$$\sum_{k:\, i \in I_k} x_{ik} = 1, \qquad a_{knn} = 1 - z_{kn} \quad (n \neq 0), \qquad \textsc{Circuit}(a_k). \tag{A.1}$$

每条被选中的非自环弧都属于经过顶点 0 的唯一回路，虚拟初始顶点的自环表示机器人闲置。被选中的弧 $a \to n$（$n \neq 0$）要求 $s_{kn} \geq f_{ka}$，并计入前往取货点或 H 的固定入向行驶。任务副本在被选中时共享 s_i, c_i，且 $s_i \geq r_i$；装货、载货行驶和卸货按等式约束依次推进时间。未激活访问的时间为零，未激活的通行与充电等待也为零。充电访问按顺序使用：$y_{k,q+1} \leq y_{kq}$，当 $y_{k,q+1} = 1$ 时 $s_{k,q+1} \geq f_{kq}$。禁止从充电访问直接连到虚拟终点的弧；从任务连到虚拟终点的闭合弧不含返程行驶。

$$e = \text{arrival} + w, \qquad l = e + \tau^{X}_{kr}, \qquad \textsc{NoOverlap}\{[e,\, l + \delta)\}, \tag{A.2}$$

$$\sum_{p} b_{kqp} = y_{kq}, \qquad u = h + w^{C} + \tau_{Hp}, \qquad f = u + D + \tau_{pH}, \tag{A.3}$$

$$S^{-} = S^{H} - \rho_w w^{C} - g_{Hp} \geq S_{\min}, \qquad S^{+} = S^{-} + \rho_c D \leq 100, \qquad S^{+} - g_{pH} \geq S_{\min}. \tag{A.4}$$

其中 τ^{X}_{kr} 为该次通行驶入 X 的边的行驶时间。资源排队只发生在 WA、WB 或 H，绝不在 X 内。充电等式只对选中的充电位生效，其可选区间加入该充电位的 `NoOverlap`。以刻度计，启用的 D 介于 1 与 $\lceil 100U/\alpha \rceil$ 之间，其中 $\alpha = \lfloor \rho_c U/T \rfloor$。在每条被选中且不含充电的路径上，$S_{\text{out}} = S_{\text{in}} - \sum g - \rho_w \sum w$（含装卸）。冻结的预约是强制的。最后一次送达后必须在安全下限之上保留前往充电位的能耗、$3(95 - S_{\min})\rho_w/\rho_c$ 以及 0.5 个百分点的余量；解码时还会检查实际可达性。

$$\min\ J = \frac{1}{4}\left(\frac{C_{\max}}{\hat{R}_C} + \frac{\sum_i p_i T_i}{\hat{R}_T} + \frac{W_X + W_C}{\hat{R}_W} + \frac{E}{\hat{R}_E} \right), \qquad C_{\max} = \max_i c_i. \tag{A.5}$$

归一化分母取 FCFS 各分项，并以 $(1, 60, 60, 1)$ 为下限；E 不计补电。滚动时域求解保留已固定的历史贡献，只用已知任务归一化。取 $T = 1000$ 刻度/秒、$U = 60\,000\,000$ SOC 单位/百分点，时长、释放时间、能耗和排队耗电向上取整，截止时间、初始 SOC 和充电增量向下取整。不在网格上的路径在每次通过交叉口或前往充电位时额外计入 20 个刻度的耗电作为模型保护；这部分计入模型中的 E，但不出现在解码后的动作中。访问时间上界为 $\bar{H} = \lceil T\,[\, t_0 + (|I| + 8)(20\,\tau_{\max} + 5000)\,] \rceil$，其中 t_0 为最晚的初始时间或释放时间，$\tau_{\max}$ 为最长的边行驶时间。机器人 k 拥有 $\max(q^{\text{hint}}_k,\, 1,\, \lfloor |I_k|/5 \rfloor)$ 个充电时段，其中 q^{hint}_k 为联合贪心初始解中机器人 k 的充电次数；显式上限也会约束回退方案。返回有效解码候选与可接受的联合贪心计划中实际 J 较低者；二者都不可用时运行失败。OPTIMAL 指的是这一有界整数模型。

B 实验数据与调度输出

B.1 输入任务列表

表 B.1 列出算例的全部 12 项任务。这些任务为合成数据，在 FCFS、CP-SAT 和滚动时域优化中保持不变；所有时间单位为秒。R1–R4 的初始 SOC 为 50.1、53.9、52.2 和 80.8 %，安全下限为 25 %，启用两个充电位。

B.2 机器人调度与充电记录

表 B.2 给出 CP-SAT 解码后每台机器人的任务顺序。“最后送达”指该机器人最后一项运输任务的完成时间；最终 SOC 和能耗针对其完整的动作轨迹。环路 A 在 1 165.2 s 完成，环路 B 在 947.4 s 完成，因此完工时间为 1 165.2 s，$J = 0.4472$，最低 SOC 为 25.9 %，无约束违反。

表 B.1: 完整输入任务列表。

任务	取货 → 送货	释放	截止	装货	卸货	优先级
T005	B4 → B3	30	835	10	20	2
T012	A4 → A2	40	547	20	15	1
T004	B1 → B2	100	989	10	15	3
T002	B4 → B3	110	729	15	10	3
T008	B2 → B3	130	911	15	15	1
T003	B4 → B3	170	748	20	10	2
T007	B2 → B3	180	1072	20	20	1
T009	A2 → A1	190	829	20	15	1
T010	A2 → A1	190	937	10	20	1
T011	A2 → A1	200	1023	15	20	2
T001	B1 → B2	240	720	15	10	1
T006	A1 → A3	280	944	10	20	3

表 B.2: 机器人任务顺序与结果（CP-SAT）。

机器人	任务顺序	最后送达（s）	最终 SOC（%）	能耗（百分点）
R1	T009, T006	984.8	42.5	33.8
R2	T012, T011, T010	1165.2	46.5	52.0
R3	T005, T004, T007	947.4	42.5	45.4
R4	T001, T008, T002, T003	602.0	51.6	29.2

表 B.3 将 R1 的充电访问展开为到达与离开、排队、充电时长和 SOC。FCFS 下 R1 在 H 排队 634.4 s；CP-SAT 则安排 R1 在充电位空出后才到达，并且只充到 52.1 %。从 H 到 C1 行驶需要 20 s。

表 B.3: R1 的充电访问记录。

策略	节点	到达 / 离开（s）	排队（s）	充电（s）	进 / 出 SOC（%）
FCFS	H	116.0 / 750.4	634.4	0	39.5 / 37.4
FCFS	C1	770.4 / 1486.2	0	715.8	35.4 / 95.0
CP-SAT	H	534.4 / 534.4	0	0	27.9 / 27.9
CP-SAT	C1	554.4 / 868.8	0	314.4	25.9 / 52.1

三种策略的完整输出共包含 36 条任务记录、372 条动作记录和 265 次途经点访问。动作记录保留机器人和任务编号、节点、开始和结束时间、能量、SOC 和资源；能量为正表示消耗，为负表示补电。60 次本地补充运行与 700 次不同的历史运行分开保存，因为二者的代码版本和预算不同。

C　关键代码

交叉口预约。 `optimizer.py` 中的 `JointModel.path` 将被选中的一次通过转换为可选区间，其长度为 $\tau^{X}_{kr}+\delta$（即代码中的 `dt + self.gap`）；未激活访问的进入时间和等待均为零。随后每种资源各用一个 `NoOverlap`，保证 X 和每个充电位的容量。

```
wait, entry = self.var('w_cross'), self.var('e_kr')
self.model.add(entry == at + wait).only_enforce_if(use)
self.model.add(entry == 0).only_enforce_if(~use)
self.model.add(wait == 0).only_enforce_if(~use)
self.resources['X'].append(self.model.new_optional_fixed_size_interval_var(
    entry, dt + self.gap, use, 'X'))
...
for intervals in self.resources.values():   # X, C1, C2
    m.add_no_overlap(intervals)
```

解码与安全回退。 CP-SAT 的解先经规划器重放和验证，启用末端储备时还需满足该要求，再按实际目标值与联合贪心初始解比较；否则保留初始解并记录原因。

```
decoded = joint.decode(solver)              # replay through Planner
build_result(decoded, base.results + tasks, 'cpsat',
             started, opts['norm'])         # independent validation
if opts['terminal_reserve']:                # robots active in this solve
    for k in active:                        # (index set abridged)
        decoded.ensure_reserve(k)           # charger reachable at end
if (selected is None or score(decoded, opts['norm'])
        <= score(selected, opts['norm']) + 1e-7):
    selected = decoded
else:
    metadata['fallback'] = 'joint hint has a lower decoded objective'
```

复现。 仓库（快照 `9bf0b70`）可在 Python 3.12 及以上版本和 OR-Tools 9.15.6755 下运行，以下命令依次运行测试、用当前代码重新运行实验设计（存档结果来自较早的若干版本，见第 6 节）以及启动浏览器回放：

```
python -m unittest discover -s tests          # 69 tests
python experiments.py --seeds 10 --time-limit 120 --out results
python app.py                                 # browser replay
```